Boolean Algebra & Logic Circuits, Part II

The Algebra of Cofactors and SAT Solvers

Chee Wei Tan

Where Part I Ended

Part I closed with a list of *further discussions*:

- Half adders and full adders
- $\rightarrow$ bitwise arithmetic operations
- $\rightarrow$ Galois field GF(2): addition is XOR, multiplication is AND
- $\rightarrow$ fault detection, parity checking
- $\rightarrow$ universality

and with Shannon's expansion, cofactors, and the Boolean difference $\partial f/\partial x = f_x \oplus f_{\bar{x}}$.

Part II picks up every arrow

The **half adder** is the thread. It is one XOR and one AND on the same two inputs, so it is at once an *adder*, a *field*, a *test instrument*, and a *universal gate*. We will see that cofactors, Boolean differences and quantifiers all fall out of it, and that a SAT solver is a machine for taking cofactors.

Outline

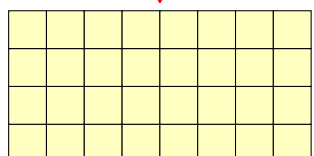
1. **Why minimize circuits?** From K-maps to a research frontier: the Circuit Minimization Team
2. **Refresher with half adders.** Cofactors of adders; the half adder as a universal gate; your first SAT puzzle
3. **The algebra of cofactors.** Shannon, Davio and Reed–Muller; Boolean difference; $\exists$ and $\forall$; unateness; a three-half-adder network that computes them all
4. **From cofactors to SAT.** CNF, Tseitin, DPLL as recursive expansion, resolution as $\exists$, CDCL
5. **SAT for synthesis.** Proving a circuit is the smallest

Recall the Wall from Part I

- A 16-bit multiplier has 32 inputs. Its K-map has $2^{32} \approx 4.3 \times 10^9$ cells.
- Grouping 1s in a K-map is a **set-cover** problem: NP-complete (Karp 1972).
- So minimization is both *combinatorially* and *computationally* hard.

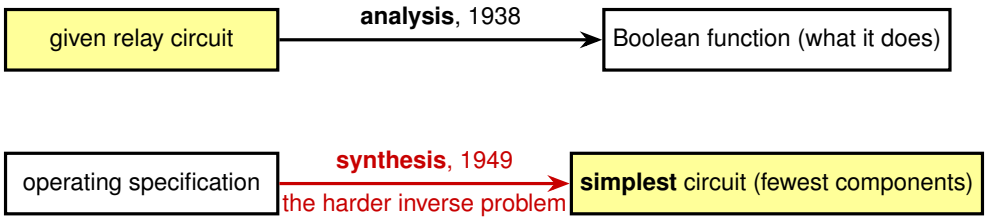
There must be a better way. The way is to stop drawing and start computing: treat f as a *data structure* and apply *operators* to it.

2^{32} cells: never by hand



a 5-variable K-map: 32 cells, still by hand

Shannon's Inverse Problem (1949)



- **1938**, *A Symbolic Analysis of Relay and Switching Circuits*: Boolean algebra describes what a given circuit computes. The half adder appears here.
- **1949**, *The Synthesis of Two-Terminal Switching Circuits*: Shannon turns the question around. Given the specification, find the network with the *minimum* number of components. By counting, almost every n-variable function needs about $2^n/n$ contacts: the first circuit lower bound. This lecture is that problem with modern tools.

What Does “Smallest Circuit” Mean? Three Cost Models

- **Gate count** $C(f)$: every gate costs 1.
- **Multiplicative complexity** $c_\wedge(f)$: count only AND gates; XOR and NOT are free.
- **XOR count** $c_\oplus(f)$: once the ANDs are fixed, how many XORs does the *linear part* need?

Why single out AND? Over GF(2) the AND is the *multiplication*. In masked hardware, multiparty computation and zero-knowledge proofs each multiplication costs randomness or communication; additions are free.

The majority function

$\mathrm{maj}(A, B, C) = AB \oplus BC \oplus CA$: three ANDs as written, but

$$\mathrm{maj} = \big((A \oplus B)(B \oplus C)\big) \oplus B$$

uses one AND. So $c_\wedge(\mathrm{maj}) = 1$ while the SOP form from Part I used three.

The Circuit Minimization Team (Boyar and Peralta)

A long-running collection of the *best known circuits* for concrete functions:

`http://cs-www.cs.yale.edu/homes/peralta/CircuitStuff/CMT.html`

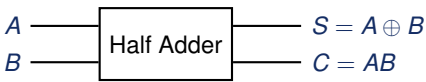
- **Two-phase method** (Boyar, Matthews, Peralta, *J. Cryptology* 2013): first reduce the AND gates, then minimize the XOR-only linear part. Even the second phase is NP-hard.
- **AES S-box**: 32 AND gates, 115 gates in total (Boyar–Peralta 2010); the reference circuit for masked AES.
- **Small functions**: all 4-input functions have $c_\wedge \leq 3$, all 5-input functions $c_\wedge \leq 4$ (Turan–Peralta 2014). Knuth: all 5-input functions need at most 12 gates.

Two phases, two algebras

- AND layer: governed by **cofactors** and the **Boolean difference** (Section 3).
- XOR layer: linear algebra over GF(2).
- The proofs that a count cannot be beaten come from **SAT solvers** (Section 5).

A half adder is an AND and an XOR bought as a pair, so it sits exactly between the two phases.

The Half Adder, Three Ways



A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

1. Arithmetic

$A + B = 2C + S$ as integers. The pair (C, S) is the two-bit number $A + B$; nothing is lost.

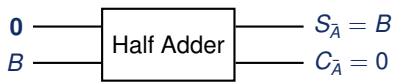
2. Field GF(2)

$S = A + B$ and $C = A \cdot B$ in GF(2). One half adder performs *both* field operations on the same operands.

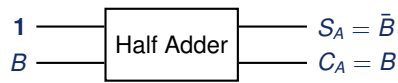
3. Cofactors (new today)

Fix one input and watch the gate collapse. Next slide.

Cofactors of a Half Adder: Fix A and Watch



$A = 0$: the XOR is a **wire**,
the AND is a **constant 0**



$A = 1$: the XOR is an **inverter**,
the AND is a **wire**

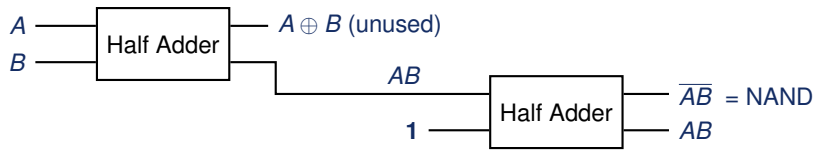
$$S_{\bar{A}} = B, \quad S_A = \bar{B}, \qquad C_{\bar{A}} = 0, \quad C_A = B.$$

Hold on to the right-hand picture. A half adder with a constant 1 is an inverter with a free copy of its input. That is the universality proof.

Half Adder as Universal Gate (Promised in Part I)

Theorem

Every Boolean function is computed by a network of half adders and the constant 1.



Proof.

The carry of $\mathrm{HA}(A, B)$ is AB; the sum of $\mathrm{HA}(AB, 1)$ is $\overline{AB}$. Two half adders make a NAND, and NAND is universal (Sheffer, Part I). □

Why the Constant Is Not Optional

Without the constant 1

A network of half adders with no constants computes only **zero-preserving** functions: $f(0, 0, \ldots, 0) = 0$.

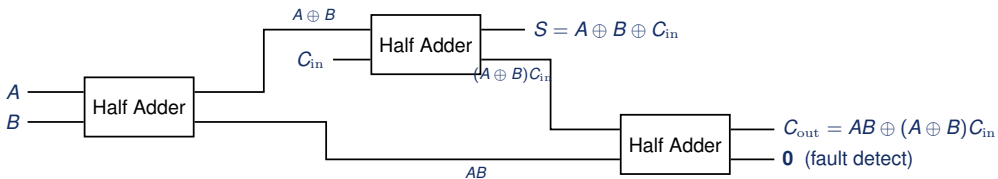
Reason. Feed all zeros in: every XOR of zeros is 0 and every AND of zeros is 0, so every wire is 0, including the outputs.

So NOT, NAND, NOR and XNOR are out of reach without a 1, while XOR, AND, OR, majority and every adder output are in reach. The constant is exactly what is missing, nothing more.

Recall Part I's third half adder

In the three-half-adder full adder, the last carry output is a constant 0 and was labeled "constant used to detect fault". A constant output that should never flip is a built-in test point: if it ever reads 1, a gate is broken.

Recall: Full Adder from Three Half Adders



Why may the final OR of two carries be an XOR? Because AB and $(A \oplus B)C_{\text{in}}$ are never both 1. In cofactor language: $(AB)_{A \oplus B} = 0$. The third half adder's carry is therefore identically 0.

Your First SAT Problem (It Was in Part I)

Part I asked: find A, B, C such that

$$AB \oplus AC \oplus BC = 1, \qquad A \oplus B \oplus C = 1.$$

Read it with a full adder

The left-hand side is C_{out} and the right-hand side is S of the full adder on (A, B, C). So the two equations say

$$A + B + C = 2 \cdot 1 + 1 = 3,$$

hence $A = B = C = 1$ is the *unique* solution.

What you just did

You asked whether a system of Boolean equations has a solution and found it. That is **satisfiability**. The solving step was not search: it was *arithmetic* on a half-adder network. Section 4 will do the same by cofactors, and show when arithmetic is not available.

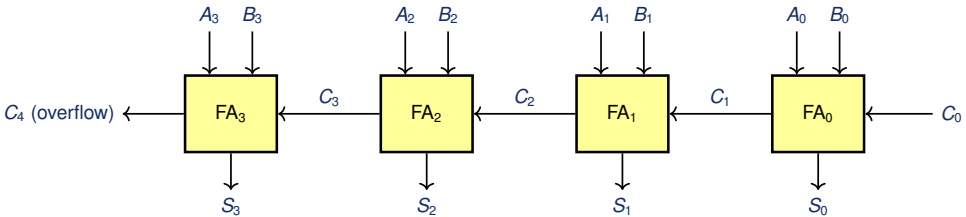
Cofactors of the Full Adder

$$S = A \oplus B \oplus C_{\text{in}}, \qquad C_{\text{out}} = \text{maj}(A, B, C_{\text{in}}) = AB + AC_{\text{in}} + BC_{\text{in}}.$$

	$A = 0$	$A = 1$
S	$B \oplus C_{\text{in}}$	$\overline{B \oplus C_{\text{in}}}$
C_{out}	$B\,C_{\text{in}}$	$B + C_{\text{in}}$

- Each cofactor is a function of **two** variables: cofactoring removes a variable.
- S: the two cofactors are complements of each other. C_{out}: one cofactor is contained in the other, $BC_{\text{in}} \leq B + C_{\text{in}}$.
- These two patterns, “complementary” and “nested”, will be named *binate* and *unate* in Section 3, and they decide how hard a function is to search.

Cofactors Across the Ripple-Carry Adder



Each stage: $C_{i+1} = G_i + P_i C_i$ with **generate** $G_i = A_i B_i$ and **propagate** $P_i = A_i \oplus B_i$.

$$(C_{i+1})_{C_i} = G_i + P_i = A_i + B_i, \qquad (C_{i+1})_{\bar{C}_i} = G_i, \qquad (C_{i+1})_{C_i} \oplus (C_{i+1})_{\bar{C}_i} = P_i.$$

Part I computed exactly this: $\partial C_{\text{out}} / \partial C_{\text{in}} = A \oplus B$. The propagate signal *is* a Boolean difference.

The Carry Chain as a Derivative, and a Test Vector

Over four stages, $C_4 = G_{3:0} + P_{3:0}\, C_0$ with $P_{3:0} = P_3 P_2 P_1 P_0$, and $G_{3:0} P_{3:0} = 0$ (if every stage propagates, none generates). Hence

$$\frac{\partial C_4}{\partial C_0} = P_3 P_2 P_1 P_0.$$

A change at C_0 reaches the overflow bit exactly when *every* stage propagates. Carry-lookahead adders are built by expanding this derivative.

Fault-tolerant adder, Part I

To test whether C_0 is **stuck at 0**, choose inputs with $\partial C_4 / \partial C_0 = 1$:

$$A = 0101, \quad B = 1010 \quad (A \oplus B = 1111),$$

apply $C_0 = 1$ and watch C_4. If $C_4 = 0$, the carry input is stuck. One Boolean difference, one test vector.

Cofactor: Definition (Recap) and Notation

Definition

For $f(x_1, \ldots, x_n)$ and a variable x:

$$f_x = f|_{x=1} \text{ (positive cofactor)}, \qquad f_{\bar{x}} = f|_{x=0} \text{ (negative cofactor)}.$$

- A cofactor is a **function**, not a number; it no longer depends on x.
- Cofactor by several literals at once: $f_{x\bar{y}} = f|_{x=1,y=0}$. Order does not matter: $(f_x)_y = (f_y)_x = f_{xy}$.
- Cofactoring by all n variables reaches a constant: a truth-table entry.

Example (Part I)

$F = xy + x\bar{z} + y(\bar{x}z + \bar{z})$: $\quad F_x = y + \bar{z} + y\bar{z} = y + \bar{z}, \quad F_{\bar{y}} = x\bar{z}.$

The One Rule That Makes Cofactoring an Algebra

Theorem (Cofactoring commutes with every gate)

For any operator $\odot \in \{\wedge, \vee, \oplus, \dots\}$ and any literal ℓ:

$$(\bar{f})_\ell = \overline{f_\ell}, \qquad (f \odot g)_\ell = f_\ell \odot g_\ell.$$

Proof.

Setting x to a constant is a pointwise substitution, and a pointwise substitution commutes with every pointwise operation. □

Part I stated these as "cofactor of AND is AND of cofactors", and so on. Today they are the engine:

cofactor first, operate second.

Example

$(S \cdot C_{\text{out}})_A = S_A \cdot (C_{\text{out}})_A =$
$\overline{B \oplus C_{\text{in}}}\,(B + C_{\text{in}}) = B\,C_{\text{in}}.$
No need to multiply out $S \cdot C_{\text{out}}$ first.

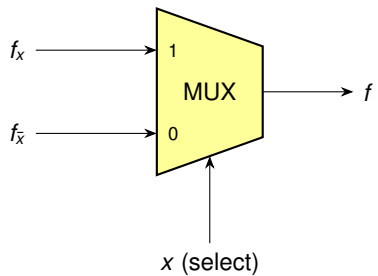
Shannon's Expansion Is a Multiplexer

Theorem (Boole; Shannon 1938)

$$f = x\, f_x + \bar{x}\, f_{\bar{x}} = x\, f_x \oplus \bar{x}\, f_{\bar{x}}.$$

The two terms are disjoint (one has x, the other $\bar{x}$), so OR and XOR agree.

Expand on every variable and you get a MUX tree with 2^n leaves: the truth table again. Sharing equal subtrees gives the **BDD** of Part I.



Part I drew the same picture as "two copies of F and a MUX".

The Same Identity in the XOR Basis: Davio Expansions

Substitute $\bar{x} = x \oplus 1$ into Shannon's expansion and collect:

$$f = x f_x \oplus (x \oplus 1) f_{\bar{x}} = f_{\bar{x}} \ \oplus \ x\,(f_x \oplus f_{\bar{x}}) \qquad \text{positive Davio}$$

$$f = f_x \ \oplus \ \bar{x}\,(f_x \oplus f_{\bar{x}}) \qquad \text{negative Davio}$$

Part I called this the *Reed–Muller* or *Fourier* form:

$$f = f_{\bar{x}} \oplus x \cdot \frac{\partial f}{\partial x}.$$

"Value at $x = 0$, plus x times the derivative": the first-order **Taylor expansion**, and over GF(2) it is *exact*.

Majority

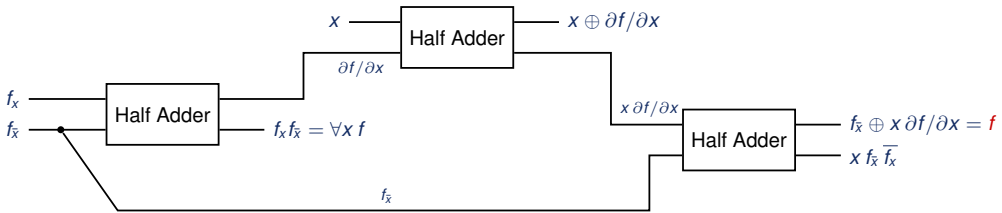
$\mathrm{maj}_{\bar{A}} = BC_{\mathrm{in}}$,
$\partial \mathrm{maj}/\partial A = (B + C_{\mathrm{in}}) \oplus BC_{\mathrm{in}} = B \oplus C_{\mathrm{in}}$, so

$$\mathrm{maj} = BC_{\mathrm{in}} \ \oplus \ A(B \oplus C_{\mathrm{in}}),$$

the C_{out} formula of the three-half-adder full adder.

Shannon's Expansion Built from Half Adders

Part I drew Shannon's expansion as an *adder* fed with x, $f_{\bar{x}}$ and f_x. Here is the precise version: three half adders.



The output f is the positive Davio expansion. The *by-products* are the Boolean difference (sum of HA_1) and the consensus $\forall xf$ (carry of HA_1). One network, three operators.

Reed–Muller Form: Functions as GF(2) Polynomials

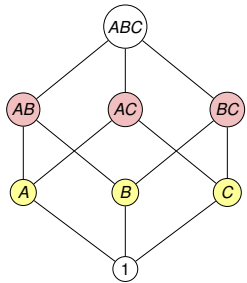
Iterate the positive Davio expansion on every variable. The result is the unique **algebraic normal form** (Zhegalkin, Reed–Muller):

$$f = \bigoplus_{T \subseteq \{1..n\}} a_T \prod_{i \in T} x_i, \qquad a_T \in \{0, 1\}.$$

Split off x: $f = h \oplus x\, g$ with h, g free of x. Then

$$f_{\bar{x}} = h, \quad f_x = h \oplus g, \quad \frac{\partial f}{\partial x} = g.$$

The Boolean difference is the *formal derivative* of the polynomial.



$\mathrm{maj} = AB \oplus AC \oplus BC$
$S = A \oplus B \oplus C$

"Reed–Muller polynomial is a path through the Hasse diagram" (Part I): a function is a set of monomials, one per node.

The Boolean Difference: Rules

Definition (Part I)

$\frac{\partial f}{\partial x} = f_x \oplus f_{\bar{x}}$, the set of inputs on which flipping x flips f. It does not depend on x.

From the commutation theorem:

$$\frac{\partial(f \oplus g)}{\partial x} = \frac{\partial f}{\partial x} \oplus \frac{\partial g}{\partial x} \qquad \text{(linear)}$$

$$\frac{\partial(fg)}{\partial x} = f_{\bar{x}} \frac{\partial g}{\partial x} \oplus g_{\bar{x}} \frac{\partial f}{\partial x} \oplus \frac{\partial f}{\partial x}\frac{\partial g}{\partial x} \qquad \text{(product)}$$

$$\frac{\partial \bar{f}}{\partial x} = \frac{\partial f}{\partial x}, \qquad \frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$$

- The product rule has a third term: this derivative is a *finite difference*, not a limit.
- $\partial f/\partial x = 0$ iff f does not depend on x at all.
- Part I: NOT gives 1, AND xy gives y, OR gives $\bar{y}$, XOR gives 1.

The Boolean Difference on a Half Adder, and Testing

$$\frac{\partial S}{\partial A} = \bar{B} \oplus B = 1, \qquad \frac{\partial C}{\partial A} = B \oplus 0 = B.$$

- The sum is sensitive to A *always*: any input pattern tests the XOR path.
- The carry is sensitive to A only when $B = 1$: to test the AND path through A you must hold $B = 1$.

Automatic test pattern generation

A stuck-at fault on wire x of a circuit computing f is **detectable** iff $\partial f / \partial x \neq 0$. A **test vector** is any solution of

$$\frac{\partial f}{\partial x} \wedge (x \neq \text{stuck value}).$$

Finding one is a *satisfiability* problem. Pentium FDIV (Part I) is what happens when such questions are not asked.

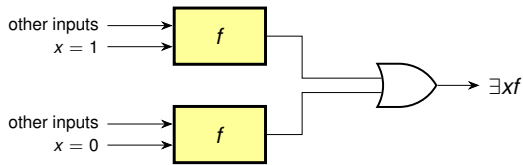
Quantifiers: Smoothing and Consensus

Definition

$$\exists x\, f = f_x + f_{\bar{x}} \qquad \text{(smoothing)}$$

$$\forall x\, f = f_x \cdot f_{\bar{x}} \qquad \text{(consensus)}$$

$\exists xf = 1$: *some* value of x makes $f = 1$.
$\forall xf = 1$: *every* value of x makes $f = 1$.
Both are free of x, and $\forall xf \leq f \leq \exists xf$.



replace OR by AND for $\forall xf$

Full adder

$\exists A\, C_{\text{out}} = (B + C_{\text{in}}) + BC_{\text{in}} = B + C_{\text{in}}$: a carry is *possible* iff some other input is 1.
$\forall A\, C_{\text{out}} = (B + C_{\text{in}})\, BC_{\text{in}} = BC_{\text{in}}$: a carry is *forced* iff both others are 1.

Three Operators, One Family

Theorem

$$\exists x\, f \;=\; \forall x\, f \;\oplus\; \frac{\partial f}{\partial x}.$$

Proof.

Write $u = f_x$, $v = f_{\bar{x}}$. Then $u + v = uv \oplus u \oplus v$ (OR in terms of XOR and AND). □

Read off the half-adder network

$\mathrm{HA}_1(f_x, f_{\bar{x}})$ outputs sum $= \partial f/\partial x$ and carry $= \forall x f$. XOR them and you get $\exists x f$. The whole family lives in one half adder.

- x is in the support of f iff $\exists x f \neq \forall x f$.
- $\overline{\exists x f} = \forall x\, \bar{f}$ (De Morgan for quantifiers).
- $\exists$ distributes over OR, $\forall$ over AND; $\exists x(fg) \leq \exists x f \cdot \exists x g$ only.

Satisfiability of F is the statement $\exists x_1 \cdots \exists x_n\, F = 1$. Tautology is $\forall x_1 \cdots \forall x_n\, F = 1$.

Unate or Binate? Majority vs Parity

Definition

f is **positive unate** in x if $f_{\bar{x}} \leq f_x$ (raising x never lowers f); **negative unate** if $f_x \leq f_{\bar{x}}$; **binate** otherwise.

Majority: unate in every input

$(C_{\text{out}})_{\bar{A}} = BC_{\text{in}} \leq B + C_{\text{in}} = (C_{\text{out}})_A$. More 1s in, more carry out. The K-map of C_{out} in Part I has its 1s in one "upper" region.

Parity: binate in every input

$S_A = \overline{S_{\bar{A}}}$: the two cofactors are complementary, neither contains the other. The K-map of S is a checkerboard: no two adjacent 1s.

When f is positive unate in x, the three operators collapse to a single cofactor:

$$\exists x f = f_x, \qquad \forall x f = f_{\bar{x}}, \qquad \frac{\partial f}{\partial x} = f_x \, \overline{f_{\bar{x}}}.$$

Tautology by Recursion: the Unate Recursive Paradigm

Is $f \equiv 1$? By Shannon's expansion,

$$f \equiv 1 \iff f_x \equiv 1 \text{ and } f_{\bar{x}} \equiv 1.$$

Pruning. If f is positive unate in x, then $f \equiv 1$ iff $f_{\bar{x}} \equiv 1$ (the smaller cofactor suffices). Split only on *binate* variables, the most binate first.

Terminal cases: a SOP containing the empty cube ($\equiv 1$), or a SOP in which some variable appears in only one polarity and no cube is empty ($\not\equiv 1$).

Example

$f = AB + \bar{A}C + \bar{B}C + \bar{C}$. All variables binate; C occurs most, so split on C:

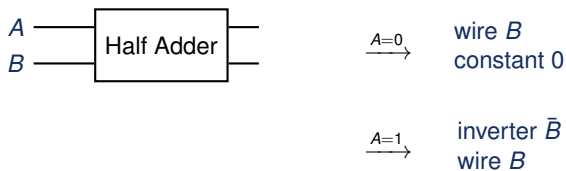
$$f_C = AB + \bar{A} + \bar{B} = AB + \overline{AB} \equiv 1,$$

$$f_{\bar{C}} = AB + 1 \equiv 1.$$

Two cofactor steps instead of an 8-row truth table.

Cofactors of Half-Adder Networks: Closure

Take any half-adder network and fix one primary input A:



An inverter is $\mathrm{HA}(B, 1)$, so the cofactor is again a half-adder network, with constants.

Proposition

Half-adder networks over $\{\mathrm{HA}, 0, 1\}$ are closed under cofactoring, and

$$\mathrm{HA}(f) \;\geq\; \mathrm{HA}(f_\ell)$$

for every literal ℓ, where $\mathrm{HA}(\cdot)$ is the least number of half adders.

This is the half-adder form of the classical *gate-elimination* argument for lower bounds: fix an input, lose a gate, induct.

Worked Example: How Many Half Adders for Majority?

Upper bound 3. $\mathrm{HA}_1(A, B)$, $\mathrm{HA}_2(A \oplus B, C_{\mathrm{in}})$, $\mathrm{HA}_3(AB, (A \oplus B)C_{\mathrm{in}})$: the full adder from Part I, whose third sum is maj.

Lower bound 3.

- One half adder gives only $\{u \oplus v,\ uv\}$ for one pair; maj depends on three inputs.
- After two half adders every signal is either linear in some input or a single product of two signals.
- $\mathrm{maj} = AB \oplus BC \oplus CA$ has degree 2 with three monomials: neither.

The same bound by cofactors

$\mathrm{maj}_A = B + C$ needs 2 half adders ($B + C = B \oplus C \oplus BC$: $\mathrm{HA}(B, C)$ then HA of its two outputs). The closure proposition gives $\mathrm{HA}(\mathrm{maj}) \geq 2$, and the degree argument lifts it to 3.

Hand arguments scale to three gates. Section 5 shows how a SAT solver produces the same “no circuit with two half adders” certificate automatically, and how it scales to the tables of the CMT.

Satisfiability

The problem

Given $F(x_1, \ldots, x_n)$, decide whether $\exists x_1 \cdots \exists x_n \, F = 1$, and if so return a **witness**.

- First NP-complete problem (Cook 1971, Levin 1973).
- Yet solvers handle millions of clauses daily: hardware verification, planning, scheduling, cryptanalysis, and *circuit synthesis*.
- Input format: **CNF**, an AND of *clauses*, each an OR of literals.

Part I's puzzle in CNF

"$\mathrm{maj}(A, B, C) = 1$": $(A + B)(A + C)(B + C)$.
"$A \oplus B \oplus C = 1$":
$(A+B+C)(A+\bar{B}+\bar{C})(\bar{A}+B+\bar{C})(\bar{A}+\bar{B}+C)$.
Seven clauses; the only satisfying assignment is $A = B = C = 1$.

Why CNF? Because Cofactoring Is Cheap There

Set a literal $\ell = 1$ in a CNF F:

- every clause containing ℓ is **satisfied**: delete it;
- every clause containing $\bar{\ell}$ **shrinks**: drop $\bar{\ell}$ from it;
- other clauses are untouched.

The cofactor F_ℓ is computed in one linear pass with no rewriting.

Example

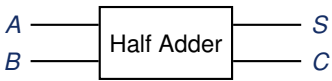
$F = (A + B)(A + C)(B + C)(\bar{A} + \bar{B} + C)$.
$F_A = (B + C)(\bar{B} + C)$.
$F_{\bar{A}} = (B)(C)(B + C)$.

Terminal cases

F_κ has **no clauses left** $\Rightarrow$ it is $\equiv 1$: SAT, witness κ. F_κ contains the **empty clause** $\Rightarrow$ it is $\equiv 0$: dead end.

(SAT of a SOP is trivial, tautology of a CNF is trivial; each form hides the hard direction of the other.)

Tseitin Encoding: A Half Adder Is Seven Clauses



Give every gate output its own variable and assert "output $=$ gate function" as clauses.

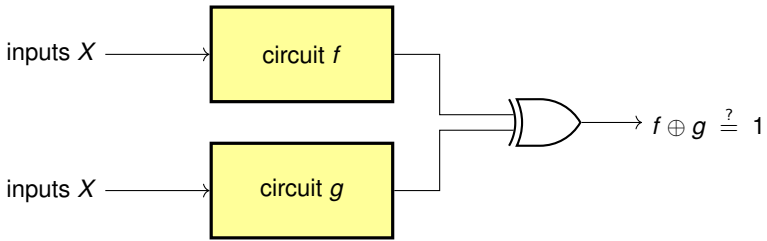
$$C \leftrightarrow A \wedge B: \quad (\bar{C} + A)(\bar{C} + B)(C + \bar{A} + \bar{B})$$

$$S \leftrightarrow A \oplus B: \quad (\bar{S} + A + B)(\bar{S} + \bar{A} + \bar{B})(S + \bar{A} + B)(S + A + \bar{B})$$

Three clauses for AND, four for XOR: seven per half adder.

- A circuit with m gates becomes $O(m)$ clauses, *equisatisfiable* with the circuit (same answer, extra variables).
- Each clause is itself a cofactor statement: $(\bar{C} + A)$ says $C_{\bar{A}} = 0$, i.e. $C \leq A$.
- A 4-bit ripple-carry adder of three-half-adder full adders: $4 \times 21 = 84$ clauses.

Equivalence Checking: the Miter



- **UNSAT** $\Rightarrow f \equiv g$. **SAT** $\Rightarrow$ the witness is an input on which they differ.
- This is how "full adder $=$ three half adders" is checked by machine, and how a synthesized 64-bit adder is checked against its specification.
- The hard answer is UNSAT: it needs a *proof*, not a witness. Everything in the rest of this section is about producing that proof fast.

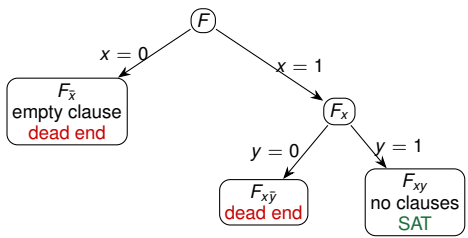
DPLL Is Recursive Shannon Expansion

Since $\exists x\, F = F_x + F_{\bar{x}}$,

$$\mathrm{SAT}(F) = \mathrm{SAT}(F_x) \;\vee\; \mathrm{SAT}(F_{\bar{x}}).$$

The search tree *is* the MUX tree of Shannon's expansion, explored depth-first: memory linear in n.

Davis, Logemann and Loveland (1962) added two rules that avoid branching. Both are cofactor identities (next slide).



The Two DPLL Rules Are Cofactor Identities

1. Unit propagation

A clause with a single literal (ℓ) forces $\ell = 1$: $F = \ell \cdot F'$, so $F_{\bar{\ell}} = 0$ and

$$\mathrm{SAT}(F) = \mathrm{SAT}(F_\ell).$$

Repeat until no unit clause remains (*Boolean constraint propagation*, BCP).

2. Pure literal

If x occurs only positively in F, then F is **positive unate** in x, so $F_{\bar{x}} \leq F_x$ and

$$\mathrm{SAT}(F) = \mathrm{SAT}(F_x).$$

This is the unateness collapse of Section 3 in CNF clothing.

Branch on a variable only when neither rule applies; choose a *binate* variable that appears often.

Worked Example: DPLL on Part I's Puzzle

$F = (A + B)(A + C)(B + C)\,(A + B + C)(A + \bar{B} + \bar{C})(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + C)$. No unit, no pure literal. Branch on A.

Branch $A = 0$:

$$F_{\bar{A}} = (B)(C)(B + C)(B + C)(\bar{B} + \bar{C})$$

- unit B: $F_{\bar{A}B} = (C)(\bar{C})$
- unit C: $F_{\bar{A}BC} = (\,)$, the empty clause. Dead end.

Branch $A = 1$:

$$F_{A} = (B + C)(B + \bar{C})(\bar{B} + C)$$

- no unit; B and C both binate; branch $B = 0$: $(C)(\bar{C})$, dead end.
- $B = 1$: (C), unit C: no clauses left. **SAT** with $(A, B, C) = (1, 1, 1)$.

Five cofactor steps; the full-adder shortcut on slide 13 took one. Section 4's last slides explain why XOR-heavy formulas make SAT do it the long way.

Resolution Is Existential Quantification

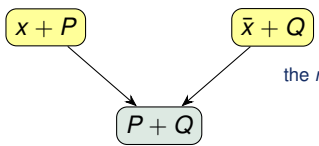
Theorem

For clauses $x + P$ and $\bar{x} + Q$ with P, Q free of x:

$$\exists x \left[(x + P)(\bar{x} + Q)\right] = P + Q.$$

Proof.

With $f = (x + P)(\bar{x} + Q)$: $f_x = Q$, $f_{\bar{x}} = P$, and $\exists x f = f_x + f_{\bar{x}}$. □



the *resolvent* on x (the *consensus* of Part I)

For a whole CNF: $\exists x F$ = (clauses without x) $\wedge$ (all resolvents on x). Eliminating a variable is quantifying it out.

Two Algorithms, Two Operators

Davis–Putnam 1960: eliminate

Apply $\exists x$ to one variable at a time by resolution until no variables remain. Complete, but the clause set can grow exponentially.

DPLL 1962: split

Replace elimination by Shannon's expansion (branching). Linear memory; exponential time in the worst case.

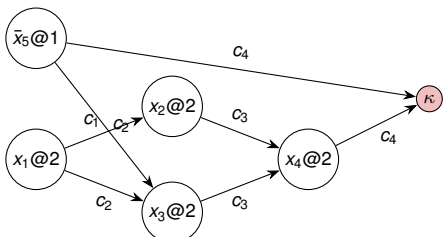
CDCL 1996–: split, and learn resolvents

Keep splitting, but at every dead end resolve a few clauses and **keep the resolvent**. The learned clauses form a resolution proof on the side; when the empty clause is derived, the formula is refuted.

GRASP (Marques-Silva and Sakallah 1996), Chaff (2001), MiniSat (2003), and today's solvers are all of this type.

CDCL: One Conflict, Step by Step

Clauses: $c_1 = (\bar{x}_1 + x_2)$, $c_2 = (\bar{x}_1 + x_3 + x_5)$, $c_3 = (\bar{x}_2 + \bar{x}_3 + x_4)$, $c_4 = (\bar{x}_4 + x_5)$.



Decision level 1: $x_5 = 0$. Level 2: $x_1 = 1$; BCP implies x_2, x_3, x_4; c_4 is falsified: conflict κ.

Conflict analysis = resolution backwards:

$$c_4 \text{ res}_{x_4} c_3 : (\bar{x}_2 + \bar{x}_3 + x_5)$$
$$\text{res}_{x_3} c_2 : (\bar{x}_1 + \bar{x}_2 + x_5)$$
$$\text{res}_{x_2} c_1 : (\bar{x}_1 + x_5)$$

Stop when one literal of the current level remains (*first UIP*): **learn** $(\bar{x}_1 + x_5)$.

Backjump to level 1, where the learned clause is a unit: $x_1 = 0$ is now forced. The solver never re-enters the whole $x_1 = 1$ subtree, under any other decisions.

What Learning Buys, in Cofactor Language

- A conflict says $F_\kappa \equiv 0$ for the current cube κ of decisions and implications.
- The learned clause is $\bar{\kappa'}$ for a **shorter** cube $\kappa' \subseteq \kappa$ with the same property. Learning is the search for small cubes on which F vanishes.
- Plain DPLL forgets κ' and may rediscover it under every other prefix; CDCL remembers it once.

The engineering around it

- **VSIDS**: branch on variables that appeared in recent conflicts.
- **Two watched literals**: BCP cost independent of clause length.
- **Restarts** and **clause deletion**: keep the learned set useful.

The learned clauses, ending in the empty clause, are a resolution refutation: the *certificate* that an exact-synthesis bound cannot be beaten.

Why XOR Hurts SAT

- Parity constraints have no short resolution refutations (Tseitin formulas on expander graphs, Urquhart 1987).
- A multiplier's CNF is XOR-rich: every full adder carries two XORs. Plain SAT equivalence checking of multipliers stalls at 16–32 bits.
- Part I's puzzle already showed it: the adder solved $\mathrm{maj} = 1,\ S = 1$ in one line of arithmetic; DPLL needed a tree.

Remedies

- Gaussian elimination inside the solver (XOR reasoning, CryptoMiniSat).
- **Algebraic verification**: gates as GF(2) polynomials, specification reduced by a Gröbner basis; XOR is just $+$.
- **Arithmetic bit level**: recover half adders from the netlist and check $A + B = 2C + S$ at each one. The half-adder identity from slide 8 is the whole proof.

Two Normal Forms, Two Solvers, One Hinge

	CNF / SAT	**ANF / algebra**
cheap	AND, OR, MUX	XOR, AND
expensive	XOR chains	OR, disjunction
expansion	Shannon (MUX)	positive Davio (Taylor)
cofactor	delete or shorten clauses	substitute in the polynomial
quantifier	resolution	formal derivative, ideal membership
decision procedure	DPLL / CDCL	Gröbner bases, linear algebra

The half adder is the hinge

One XOR and one AND on the same operands: it can be read in the left column (two gates, seven clauses) or in the right column (one addition and one multiplication in GF(2)) without any translation. That is why arithmetic verifiers look for half adders first.

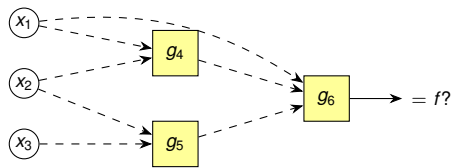
Exact Synthesis: "Is There a Circuit with k Gates?"

Kojevnikov, Kulikov and Yaroslavtsev (2009); Knuth (TAOCP 7.1.2); Haaswijk, Soeken, Mishchenko and De Micheli (2020).

Variables describing a *candidate* circuit with gates $g_{n+1}, \ldots, g_{n+k}$:

- **selection** s_{ijl}: gate i reads signals $j < l < i$;
- **function** $\phi_i(b, c)$: gate i's truth table (4 bits);
- **simulation** v_{it}: value of gate i on input row t.

Clauses: $v_{it} = \phi_i(v_{jt}, v_{lt})$ whenever s_{ijl}, for every row t; the last gate matches the target on every row.



dashed edges: chosen by s_{ijl}
gate functions: chosen by ϕ_i

Minimality Is the UNSAT Call

- Solve at $k = 1, 2, 3, \ldots$
- The first **SAT** answer returns a circuit (read it off the s and ϕ variables).
- The **UNSAT** answer at $k - 1$ is the *proof* that no smaller circuit exists. Its resolution refutation is the certificate.
- Symmetry breaking (order gates, forbid unused gates, canonical operand order) matters more than the choice of solver.

Numbers obtained this way

- Every 4-input function: at most 7 gates.
- Every 5-input function: at most 12 gates (Knuth).
- Multiplicative complexity: ≤ 3 for $n = 4$, ≤ 4 for $n = 5$ (Turan–Peralta).

These are the entries in the CMT tables.

SAT vs optimization

SAT stops at the first witness; optimization must exhaust the alternatives. The trick is to turn “minimize” into a short sequence of SAT calls whose last answer is UNSAT.

Exact Synthesis in the Half-Adder Basis

Restrict the candidate to half adders: each "gate" has *two* outputs with fixed functions,

$$v_{it}^{S} = v_{jt} \oplus v_{lt}, \qquad v_{it}^{C} = v_{jt} \wedge v_{lt},$$

and operands may be any earlier signal, including both outputs of earlier half adders. No ϕ variables: the pairing is built in.

Equivalently: breadth-first search over *sets of available signals*. A move adds $(u \oplus v,\ uv)$ for a pair in the set; stop at the first depth where the target is present. Input permutations that fix the target collapse the state space.

target	$c_\wedge$	min. half adders
$A \oplus B \oplus C$	0	2
$\mathrm{maj}(A, B, C)$	1	3
full adder (S, C_{out})	1	3
$A + B + C$ (3-input OR)	2	4
MUX $A\,?\,B : C$	1	3
2×2 multiplier	4	6

The gap between the columns is the price of buying XOR and AND in pairs. The MUX row is the three-half-adder Shannon network of Section 3.

Closing the Loop: Majority, by Hand and by Machine

By hand (Section 3): three half adders suffice, and the degree argument shows two do not.

By machine: encode "two half adders over $\{A, B, C\}$ produce maj": 2 gates, $\binom{3}{2} + \binom{5}{2}$ operand choices, 8 truth-table rows, a few hundred clauses. The solver answers UNSAT in microseconds and emits a resolution refutation. At $k = 3$ it returns Part I's full adder or an input permutation of it.

Where this stands

- Hand arguments reach $k \approx 3$.
- SAT reaches every 5-input function, and the tables of the CMT.
- Neither reaches the AES S-box: its 32-AND circuit is the best *known*, with no proof of optimality. The gap between "best found" and "provably best" is open research, and it is exactly the SAT-vs-optimization gap.

Summary: One Operator, Many Algorithms

Cofactor identity	Name	What it powers
$f = xf_x + \bar{x}f_{\bar{x}}$	Shannon expansion	MUX trees, BDDs, DPLL branching
$f = f_{\bar{x}} \oplus x\,\partial f/\partial x$	positive Davio	Reed–Muller form, algebraic verification
$f_x \oplus f_{\bar{x}}$	Boolean difference	testing, carry propagate, sensitivity
$f_x + f_{\bar{x}}$	$\exists x$ (smoothing)	SAT, resolution
$f_x \cdot f_{\bar{x}}$	$\forall x$ (consensus)	tautology, URP
$f_{\bar{x}} \leq f_x$	unateness	pure-literal rule, URP pruning
$F_{\vec{\ell}} = 0$ for a unit clause	unit propagation	BCP, implication graphs
$\exists x[(x + P)(\bar{x} + Q)] = P + Q$	resolution	Davis–Putnam, clause learning, UNSAT proofs

- One half adder on $(f_x, f_{\bar{x}})$ outputs $\partial f/\partial x$ and $\forall xf$; their XOR is $\exists xf$; two more half adders give f back.
- A SAT solver is a cofactor engine with memory. Its UNSAT answers turn “smallest circuit we found” into “smallest circuit there is”.

Exercises

1. Compute all six cofactors of $C_{\text{out}} = \text{maj}(A, B, C_{\text{in}})$ and verify $\exists A\, C_{\text{out}} = \forall A\, C_{\text{out}} \oplus \partial C_{\text{out}}/\partial A$.
2. Prove the product rule for the Boolean difference from the commutation theorem. Find the rule for $\partial(f + g)/\partial x$.
3. Show that f is linear over GF(2) (an XOR of variables, possibly with 1) iff $\partial f/\partial x$ is constant for every x.
4. Build the three-half-adder Shannon network for $f = \text{maj}(A, B, C)$ and $x = A$; label every wire.
5. Write the 21 Tseitin clauses of the three-half-adder full adder. Add the miter against maj for C_{out} and show by unit propagation alone that it is UNSAT once $A = B = 1$ is assumed.
6. Prove that $\exists x F$ for a CNF F equals the x-free clauses plus all resolvents on x. Where does the proof use that F is a CNF?
7. Run CDCL on $c_1, \ldots, c_4$ of slide 40 with the decisions $x_1 = 1$ first and $x_5 = 0$ second. What is learned now?
8. (Project) Implement the breadth-first half-adder search for three inputs and reproduce the table of Section 5. Then allow the constant 1 and compute the minimum for $\overline{\text{maj}}$ and for NAND_3.

References

- C. E. Shannon, “A symbolic analysis of relay and switching circuits,” *Trans. AIEE* 57, 1938; “The synthesis of two-terminal switching circuits,” *Bell System Technical Journal* 28(1), 59–98, 1949.
- J. Boyar, P. Matthews, R. Peralta, “Logic minimization techniques with applications to cryptology,” *J. Cryptology* 26(2), 2013; J. Boyar, R. Peralta, “A new combinational logic minimization technique with applications to cryptology,” SEA 2010; Circuit Minimization Team, `http://cs-www.cs.yale.edu/homes/peralta/CircuitStuff/CMT.html`.
- J. Boyar, R. Peralta, “Tight bounds for the multiplicative complexity of symmetric functions,” *TCS* 396, 2008; M. S. Turan, R. Peralta, “The multiplicative complexity of Boolean functions on four and five variables,” LightSec 2014.
- D. E. Knuth, *The Art of Computer Programming*, Vol. 4A §7.1.2 (Boolean evaluation); Vol. 4 Fasc. 6 §7.2.2.2 (Satisfiability).
- M. Davis, H. Putnam, “A computing procedure for quantification theory,” *JACM* 1960; M. Davis, G. Logemann, D. Loveland, “A machine program for theorem proving,” *CACM* 1962.
- J. P. Marques-Silva, K. A. Sakallah, “GRASP: a search algorithm for propositional satisfiability,” *IEEE Trans. Computers* 1999; M. Moskewicz et al., “Chaff: engineering an efficient SAT solver,” DAC 2001; N. Eén, N. Sörensson, “An extensible SAT-solver,” SAT 2003.
- R. E. Bryant, “Graph-based algorithms for Boolean function manipulation,” *IEEE Trans. Computers* 1986; “On the complexity of VLSI implementations and graph representations of Boolean functions with application to integer multiplication,” 1991.
- A. Kojevnikov, A. S. Kulikov, G. Yaroslavtsev, “Finding efficient circuits using SAT-solvers,” SAT 2009; W. Haaswijk, M. Soeken, A. Mishchenko, G. De Micheli, “SAT-based exact synthesis: encodings, topology families, and parallelism,” *IEEE TCAD* 2020.
- A. Urquhart, “Hard examples for resolution,” *JACM* 1987. G. D. Hachtel, F. Somenzi, *Logic Synthesis and Verification Algorithms*, Springer. C. Carlet, *Boolean Functions for Cryptography and Coding Theory*, CUP 2020.